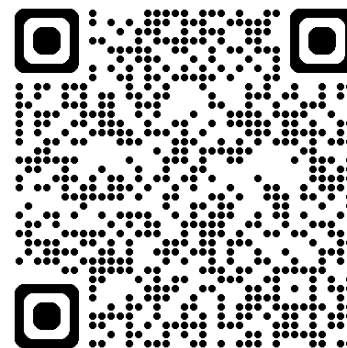




# 深度學習 Deep Learning

大模型時代如何有效率  
訓練模型？

Instructor: 林英嘉 (Ying-Jia Lin)  
2025/11/26



[Course GitHub](#)



[Slido # DL1126](#)

# Outline

---

- 輕量化訓練大型模型的方法 (50 min)
  - Parameter-efficient fine-tuning (PEFT)
- 模型壓縮
- Code (40 min)
- Homework 4 (30 min)
- Project announcements (10 min)



# The Revolution of ChatGPT

ChatGPT came out in November, 2022.

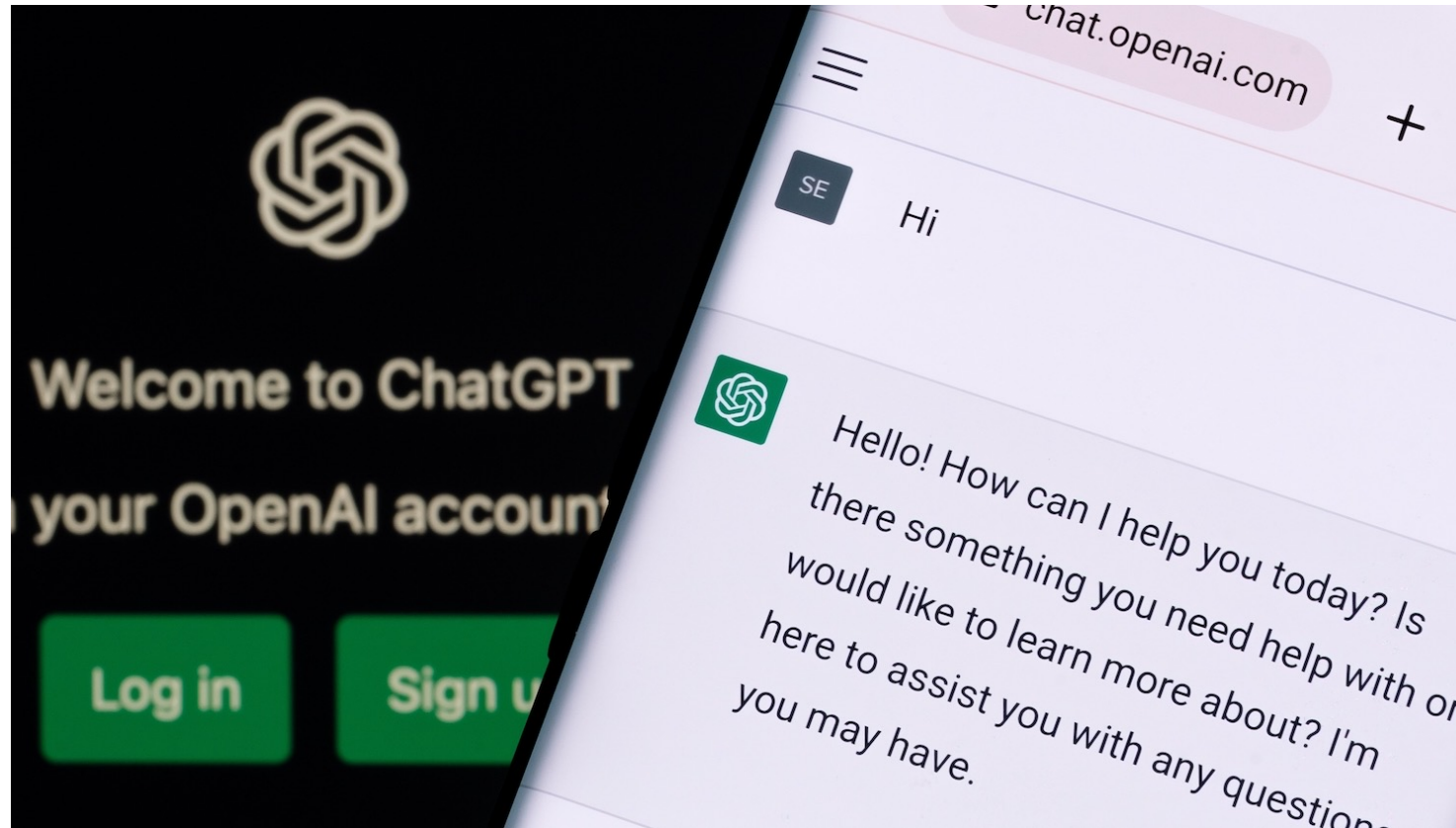
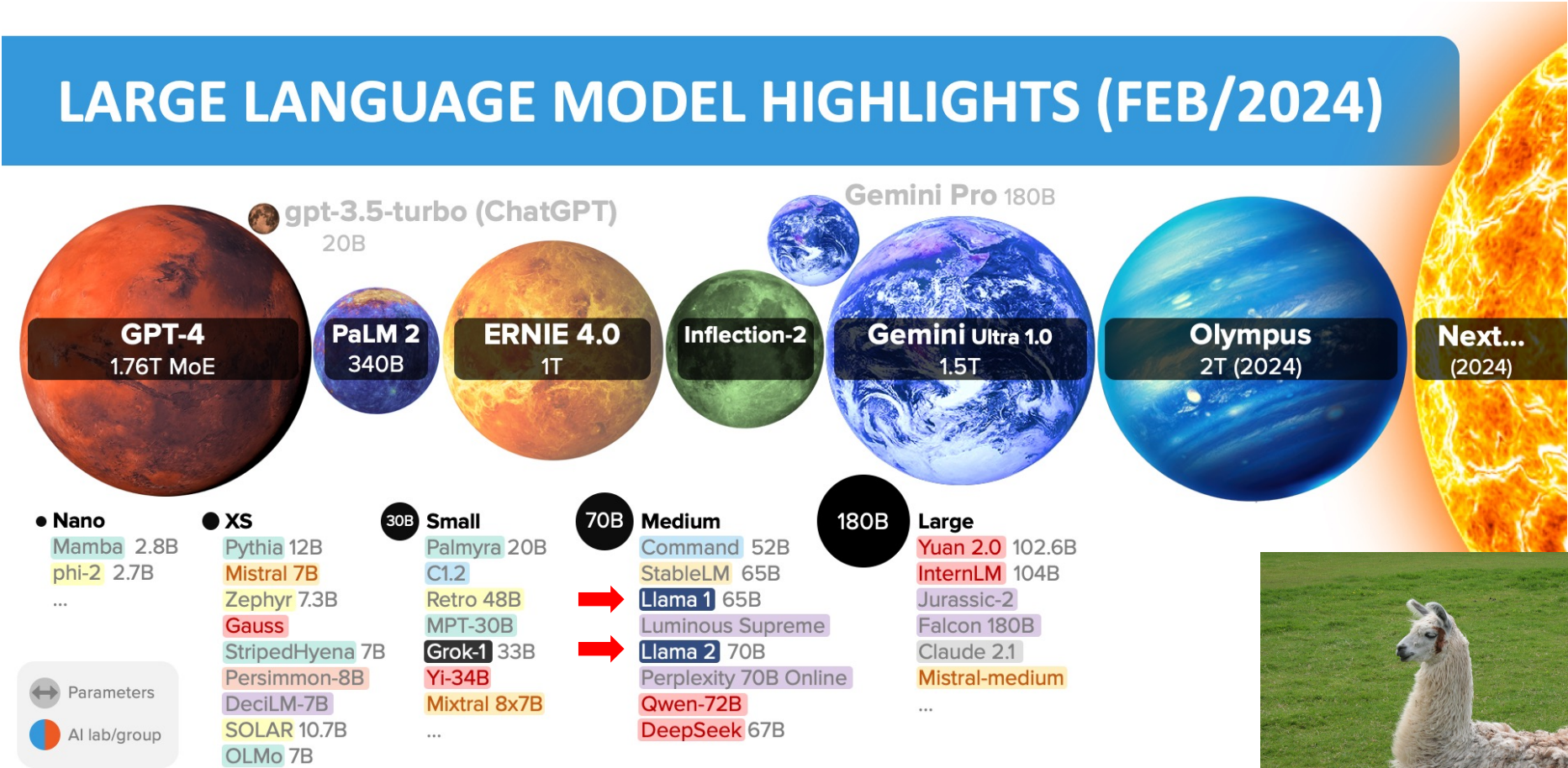


Figure source: <https://poole.ncsu.edu/thought-leadership/article/lets-chat-about-chatgpt/>  
<https://openai.com/index/chatgpt/>



# 大型語言模型參數量非常多



Sizes linear to scale. Selected highlights only. All models are available. All models are Chinchilla-aligned (20:1 tokens:parameters) <https://lilearchitect.ai/chinchilla/> All 200+ models: <https://lilearchitect.ai/models-table/> Alan D. Thompson, 2023-2024.

Source: Inside language models (from GPT-4 to PaLM) – Dr Alan D. Thompson – Life Architect



Source: [https://commons.wikimedia.org/wiki/File:Llama\\_lying\\_down.jpg](https://commons.wikimedia.org/wiki/File:Llama_lying_down.jpg)

# GPU Memory Estimated (Model weights)

---

以 Llama 2-7B 為例，且儲存參數的精度為 float 32 (FP32)：

在 float 32 (FP32) 的情況下，1 個參數需要 4 bytes 儲存

-> 7B = 70億個參數 =  $7,000,000,000 * 4 \text{ bytes}$  = (扣掉3個逗號從B換成GB) **28GB**



# 數值範圍比較

1 個 byte = 4 個 bits

| 數值類型名稱  | PyTorch dtype | 數值範圍           | Bit數 | Byte數 |
|---------|---------------|----------------|------|-------|
| float32 | torch.float32 | 約 $\pm 3.4e38$ | 32   | 4     |
| float16 | torch.float16 | 約 $\pm 6.5e4$  | 16   | 2     |
| int8    | torch.int8    | -128 ~ 127     | 8    | 1     |
| uint8   | torch.uint8   | 0 ~ 255        | 8    | 1     |
| int4    | x             | -8 ~ 7         | 4    | 0.5   |
| uint4   | x             | 0 ~ 15         | 4    | 0.5   |

u: unsigned (無號，代表數值只有正的)



# Introduction to Quantization (量化)

(這頁只是示意圖，數值不精準)

$$\begin{bmatrix} -0.4 & 1.3 & 3.73 \\ -4.7 & -3.2 & -6.4 \\ 8.5 & 14.3 & 13.5 \end{bmatrix} \xrightarrow[\text{FP32} \rightarrow \text{int8}]{\text{降低儲存精度}} \begin{bmatrix} 0 & 1 & 4 \\ -5 & -3 & -6 \\ 9 & 14 & 14 \end{bmatrix}$$

**36 bytes** **8 bytes**

32-bit floating point (FP32): 1  
個值需要**4個bytes**才能儲存

8-bit Integer (int8): 1個值需要  
**1個bytes**才能儲存

誤差：

$$\begin{bmatrix} 0.4 & -0.3 & 0.27 \\ -0.3 & 0.2 & 0.4 \\ 0.5 & -0.3 & 0.5 \end{bmatrix}$$





# 量化後的模型非常常見

<https://huggingface.co/unsloth/Kimi-K2-Thinking-GGUF>

 **Hugging Face**

[Models](#) [Datasets](#) [Spaces](#) [Community](#) [Docs](#) [Enterprise](#) [Pricing](#)

 **unsloth/Kimi-K2-Thinking-GGUF** [like 98](#) [Follow](#) [Unsloth AI](#) [10.8k](#)

[Transformers](#) [GGUF](#) [unsloth](#) [kimi\\_k2](#) [conversational](#) [License: modified-mit](#)

[Model card](#) [Files and versions](#) [xet](#) [Community 5](#) [Deploy](#) [Use this model](#)

[Edit model card](#)

### Read our How to Run Kimi-K2 Guide!

Nov 8: We collabed with the Kimi team on a [system prompt fix](#).

*Unsloth Dynamic 2.0 achieves superior accuracy & outperforms other leading quants.*

 **unsloth** [Join our Discord](#) [Documentation](#)

It is recommended to have 247 GB of RAM to run the 1-bit Dynamic GGUF.

To run the model in full precision, you can use 'UD-Q4\_K\_XL', which requires 646 GB RAM.

Downloads last month  
**40,732**



[GGUF](#) [Model size](#) [1T params](#) [Architecture](#) [deepseek2](#) [Chat template](#)

Hardware compatibility [Apple M3 \(16 GB\) x1](#)

|        |                                  |                                   |                                  |                                  |                                  |                                 |                                  |
|--------|----------------------------------|-----------------------------------|----------------------------------|----------------------------------|----------------------------------|---------------------------------|----------------------------------|
| 1-bit  | <a href="#">IQ1_S   285 GB</a>   | <a href="#">TQ1_0   247 GB</a>    | <a href="#">IQ1_M   309 GB</a>   |                                  |                                  |                                 |                                  |
| 2-bit  | <a href="#">IQ2_XXS   335 GB</a> | <a href="#">Q2_K   374 GB</a>     | <a href="#">IQ2_M   354 GB</a>   | <a href="#">Q2_K_L   374 GB</a>  | <a href="#">Q2_K_XL   386 GB</a> |                                 |                                  |
| 3-bit  | <a href="#">IQ3_XXS   422 GB</a> | <a href="#">Q3_K_S   443 GB</a>   | <a href="#">Q3_K_M   490 GB</a>  | <a href="#">Q3_K_XL   455 GB</a> |                                  |                                 |                                  |
| 4-bit  | <a href="#">IQ4_XS   547 GB</a>  | <a href="#">Q4_K_S   583 GB</a>   | <a href="#">IQ4_NL   579 GB</a>  | <a href="#">Q4_0   581 GB</a>    | <a href="#">Q4_1   643 GB</a>    | <a href="#">Q4_K_M   621 GB</a> | <a href="#">Q4_K_XL   646 GB</a> |
| 5-bit  | <a href="#">Q5_K_S   707 GB</a>  | <a href="#">Q5_K_M   729 GB</a>   | <a href="#">Q5_K_XL   730 GB</a> |                                  |                                  |                                 |                                  |
| 6-bit  | <a href="#">Q6_K   843 GB</a>    | <a href="#">Q6_K_XL   879 GB</a>  |                                  |                                  |                                  |                                 |                                  |
| 8-bit  | <a href="#">Q8_0   1.09 TB</a>   | <a href="#">Q8_K_XL   1.19 TB</a> |                                  |                                  |                                  |                                 |                                  |
| 16-bit | <a href="#">BF16   2.05 TB</a>   |                                   |                                  |                                  |                                  |                                 |                                  |

Kimi K2: Open Agentic  
Intelligence





# GPU Memory Estimated (Model weights)

以 Llama 2-7B 為例，且儲存參數的精度為 float 32 (FP32)：

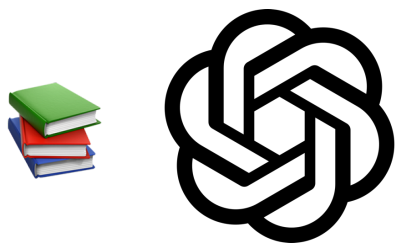
在 float 32 (FP32) 的情況下，1 個參數需要 4 bytes 儲存

-> 7B = 70億個參數 =  $7,000,000,000 * 4 \text{ bytes}$  = (扣掉3個逗號從B換成GB) **28GB**

| 模型            | 參數量  | Memory (FP32) | Memory (FP16) |
|---------------|------|---------------|---------------|
| DeepSeek v3   | 685B | 2740 GB       | 1370 GB       |
| Llama 4 Scout | 109B | 436 GB        | 218 GB        |
| GPT-2 XL      | 1.5B | 6 GB          | 3 GB          |



# 讓大型語言模型適用於你的任務？



翻譯、聊天、寫故事 ...



醫學資料、少數語言、網路上查不到的知識



解決方案：增加提示詞 或 訓練語言模型

Figure source:  
[https://gameofthrones.fandom.com/wiki/High\\_Valyrian?file=Nekesse\\_Valyrio.jpg](https://gameofthrones.fandom.com/wiki/High_Valyrian?file=Nekesse_Valyrio.jpg)

# GPU Memory Estimated (Full Fine-tuning)

| Llama 2-7B                                                                     |                                                                    |             |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------|
| 16-bit float, max_length (seq) = 4096, hidden_size = 4096, batch_size (bs) = 1 |                                                                    |             |
|                                                                                | 算法                                                                 | Memory (概略) |
| CUDA                                                                           | -                                                                  | 1 GB        |
| Model weights                                                                  | $\text{size}(\text{float}) * N_{\text{parameter}}$                 | 14 GB       |
| Gradients                                                                      | $\text{size}(\text{float32}) * N_{\text{trainable}}$               | 28 GB       |
| Hidden states                                                                  | $\text{size}(\text{float}) * \text{seq} * \text{hidden\_size} * L$ | 1 GB        |
| Optimizer states<br>(Adam)                                                     | $2 * \text{size}(\text{float32}) * N_{\text{trainable}}$           | 56 GB       |

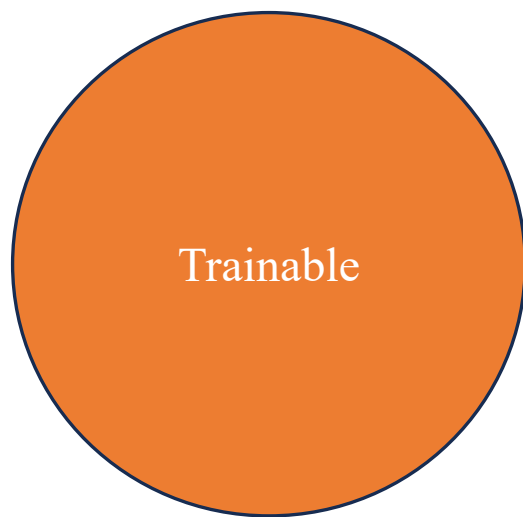
L : Number of layers in model (eq. 32 layers)

**Estimate: ~100 GB**  
\*NVIDIA 5090: 32GB

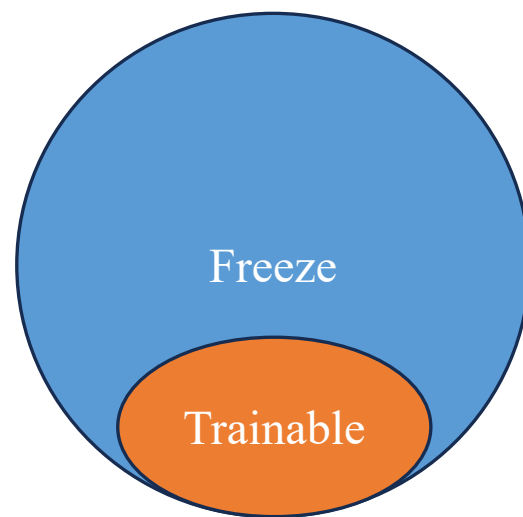


# 有沒有可能只微調 LLM 一部分的參數呢？

---



Full Fine-tuning  
Update **all model parameters**



Parameter-efficient Fine-tuning (PEFT)  
Update a **small subset** of model parameters



# GPU Memory Estimated (PEFT)

- 假設我們只更新 2% 的參數

| Llama 2-7B<br>16-bit float, max_length (seq) = 4096, hidden_size = 4096, batch_size (bs) = 1 |                                                                  |                               |
|----------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------|
|                                                                                              | 算法                                                               | Memory                        |
| CUDA                                                                                         | -                                                                | 1 GB                          |
| Model weights                                                                                | $\text{size(float)} * N_{\text{parameter}}$                      | 14 GB                         |
| Gradients                                                                                    | $\text{size(float32)} * N_{\text{trainable}}$                    | $28 * 0.02 = 0.56 \text{ GB}$ |
| Hidden states                                                                                | $\sim \text{size(float)} * \text{seq} * \text{hidden\_size} * L$ | 1 GB                          |
| Optimizer states<br>(Adam)                                                                   | $2 * \text{size(float32)} * N_{\text{trainable}}$                | $56 * 0.02 = 1.12 \text{ GB}$ |

L : Number of layers in model (eq. 32 layers)

**Estimate: 17.68 GB**

\*NVIDIA 3090: 24 GB



# PEFT Outline

---

[Adapters](#)

[LoRA](#)

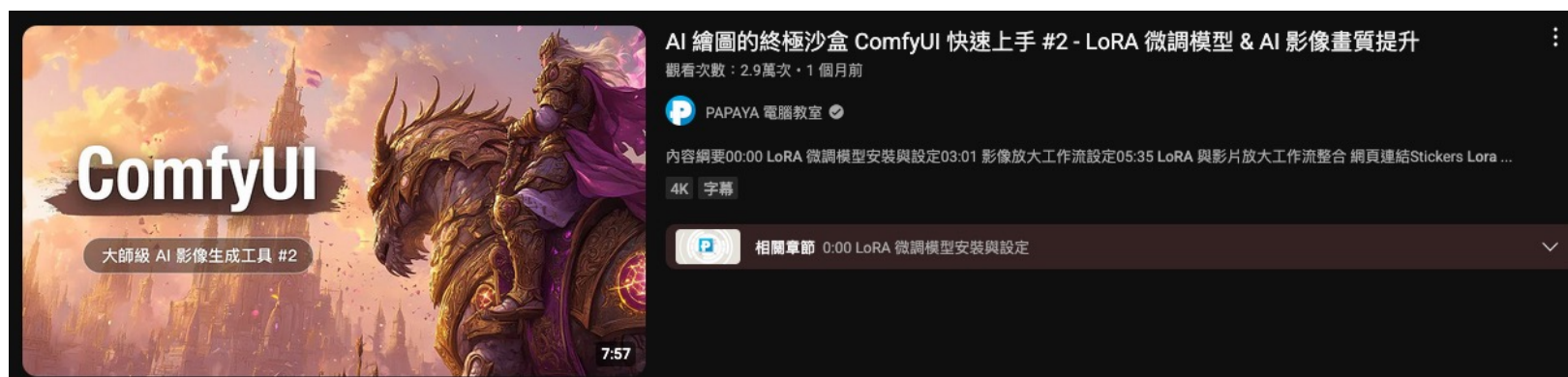
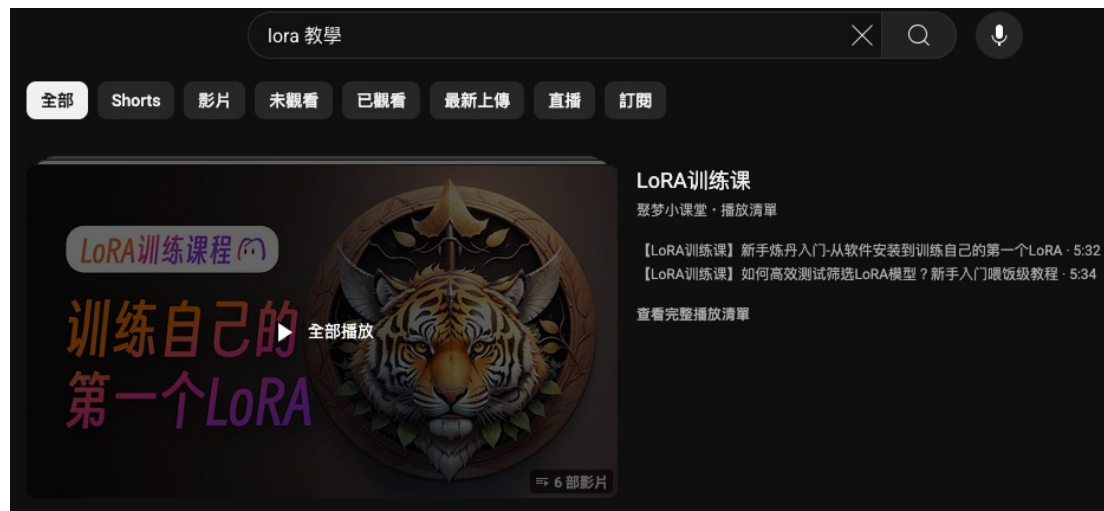
[Prefix-Tuning](#)

[BitFit](#)

LoRA: **Lo**w-**R**ank **A**daptation

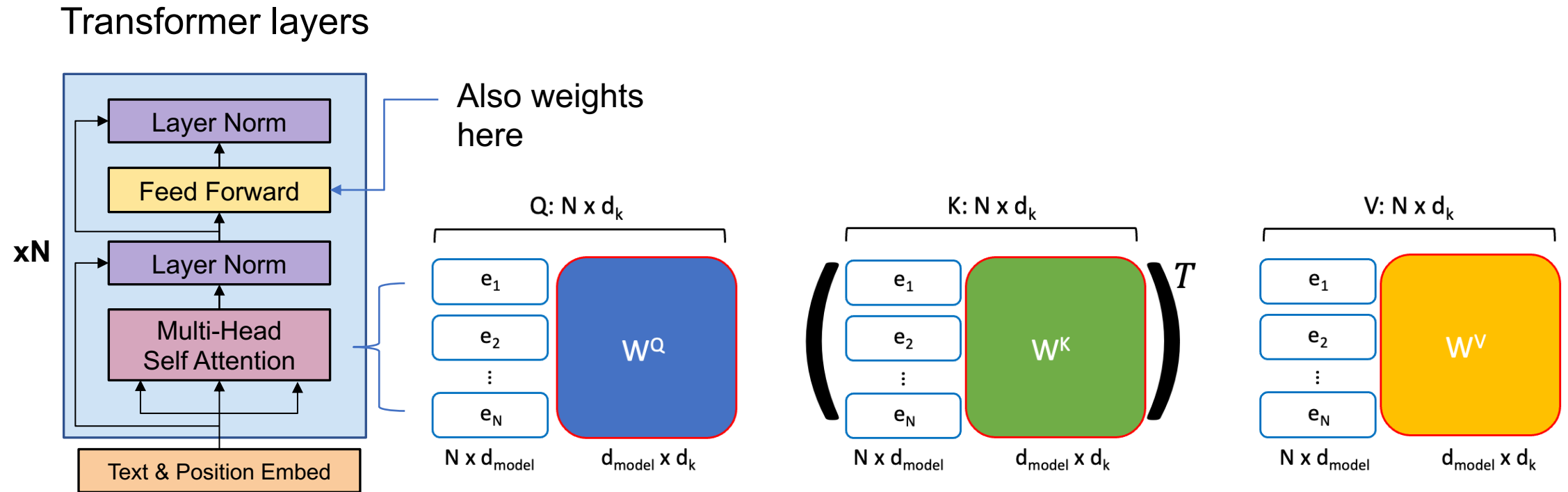


# LoRA is common ...





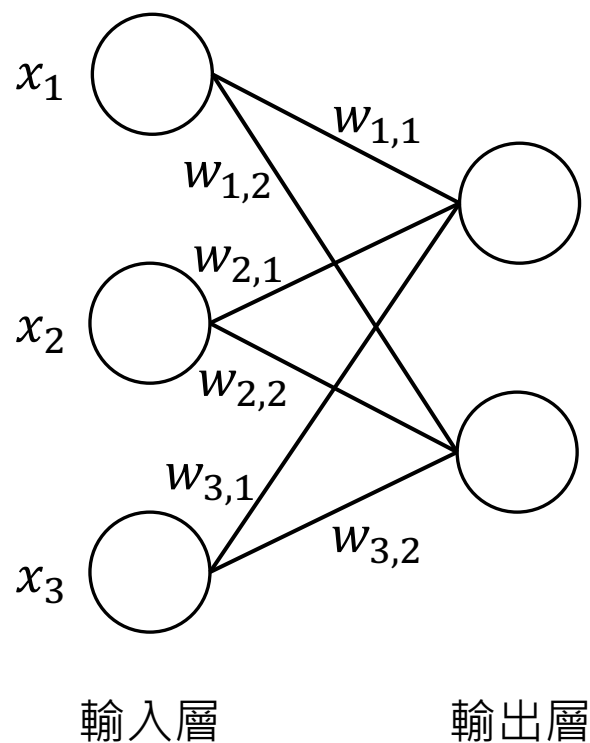
# [Prerequisite] Weights in Transformer layers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



# [Recap] MLP is composed of weight matrices



$$\begin{matrix} [x_1 & x_2 & x_3] \\ \mathbb{R}^{1 \times 3} \end{matrix} \times \begin{matrix} \boxed{\begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix}} \\ \mathbb{R}^{3 \times 2} \end{matrix} + \text{bias} \quad \mathbb{R}^2$$



# [Prerequisite] 矩陣分解



$$d = 1000$$

參數量

$$1000 \times 1000 = 1 \text{百萬}$$



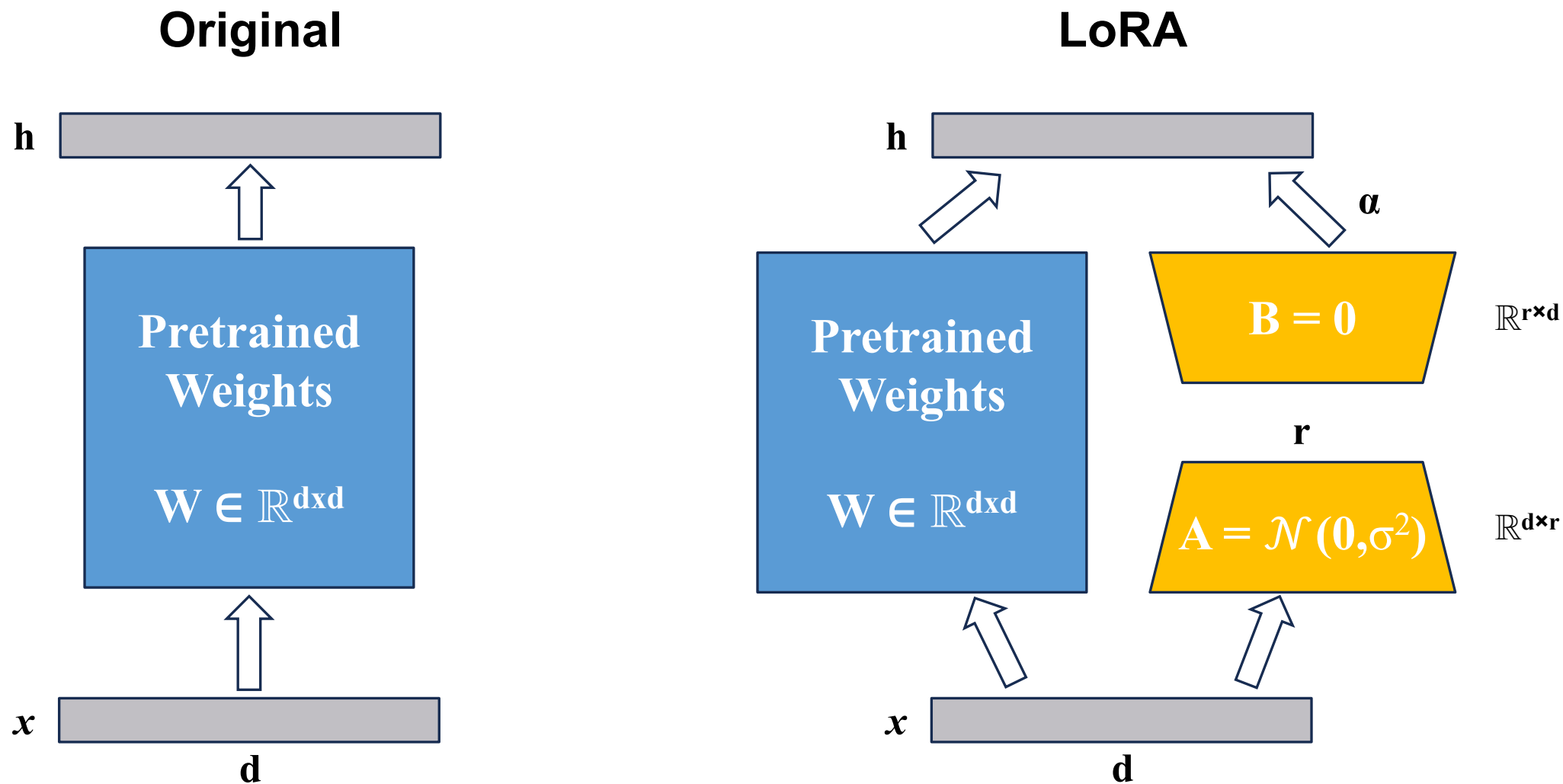
$$r = 10$$

$$1000 \times 10 + 10 \times 1000 = 20000$$

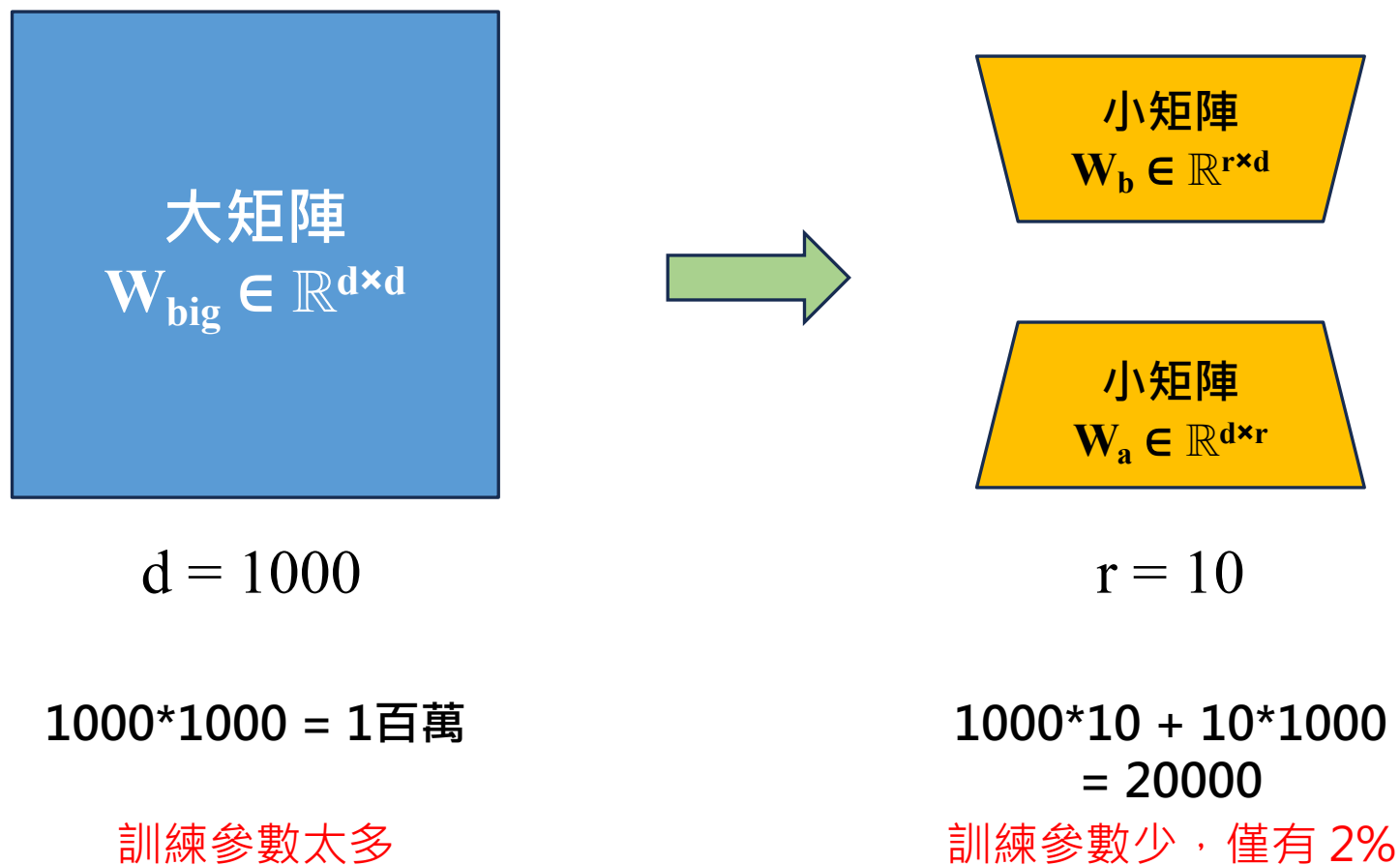


# LoRA: Low-Rank Adaptation

只有黃色部分會被訓練  
 $\alpha$  是超參數



# 為什麼 LoRA 要這樣做？



# Low-Rank 的部分在哪？(1)

$$\begin{bmatrix} \checkmark & \checkmark & & \checkmark \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Reduced row-  
echelon form

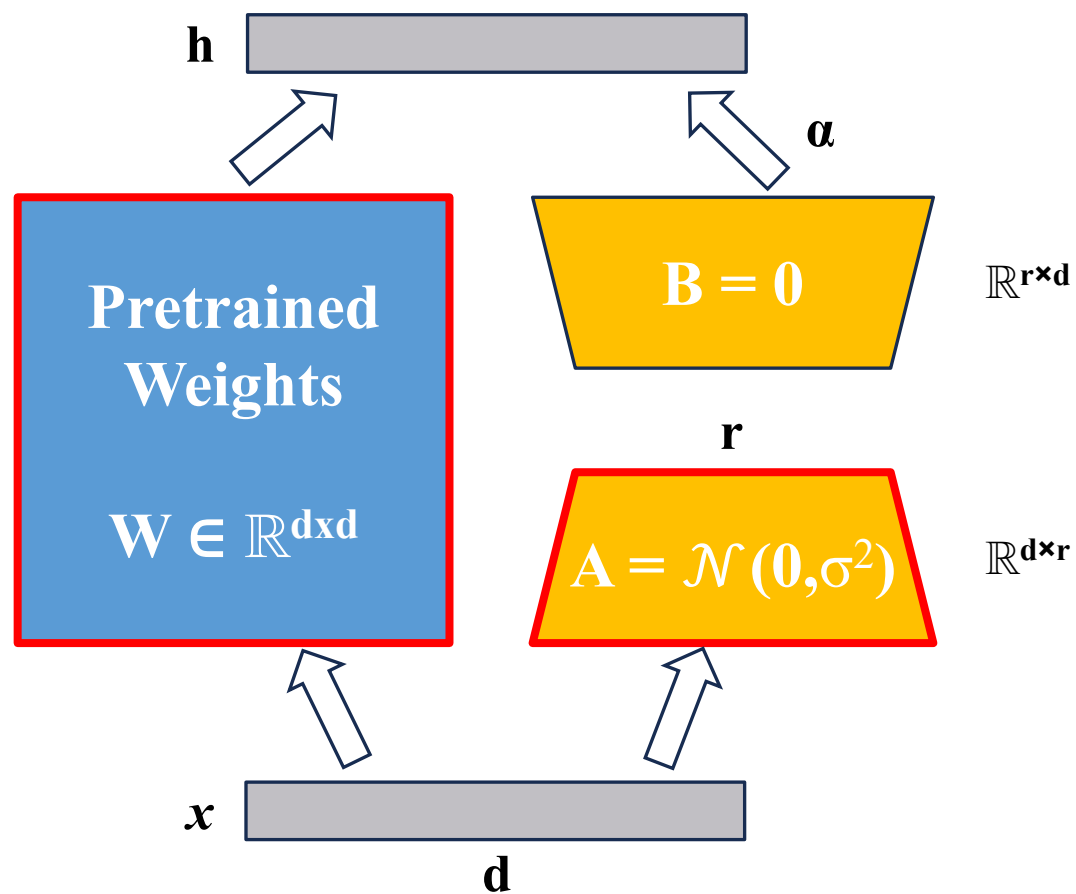
$\checkmark$  : Linearly-independent vectors  
rank = 3

rank 大小最多等於 column  
vectors (或 row vectors) 的數量



# Low-Rank 的部分在哪？(2)

## LoRA



$d > r$  的情況下 (通常  $d \gg r$ )，**A** 或 **B** 的 **rank** 通常遠小於 **W** 的 **rank** 故為 low-rank 的由來



# LoRA: **Low-Rank** **A**daptation (w / pseudo code)

Pseudocode:

```
input_dim = 768 # the hidden size of the pre-trained model
output_dim = 768 # the output size of the layer
rank = 8 # The rank 'r' for the low-rank adaptation
```

```
W = ... # from pretrained network with shape input_dim x output_dim
```

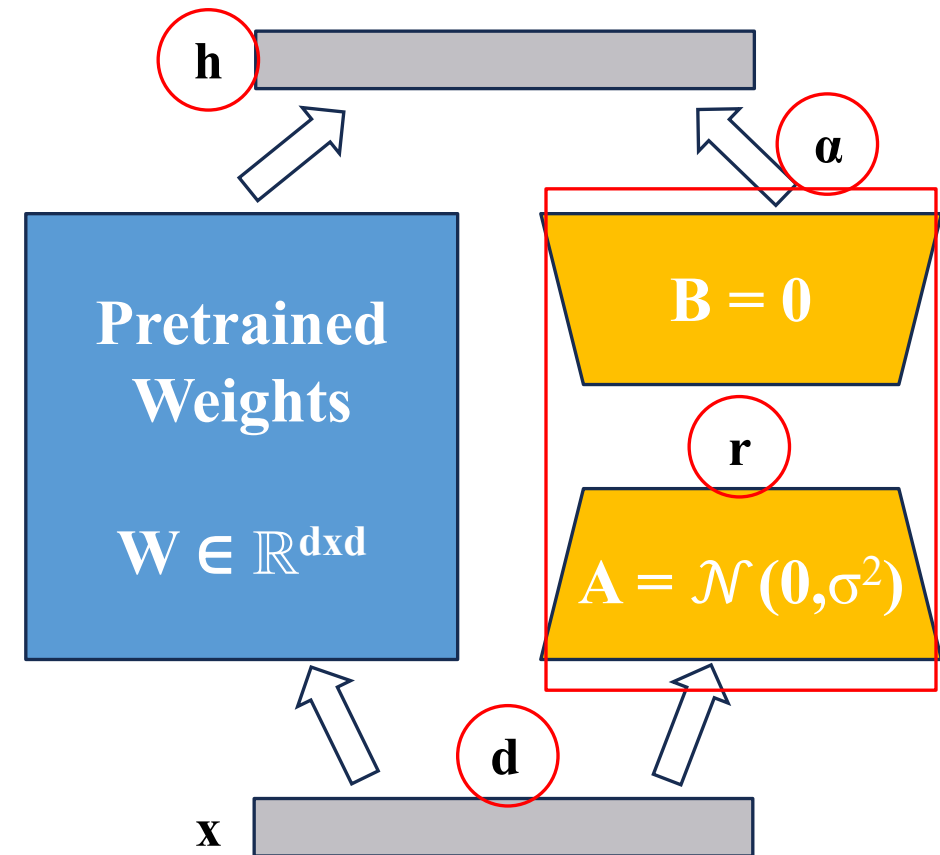
```
W_A = nn.Parameter(torch.empty(input_dim, rank)) # LoRA weight A
W_B = nn.Parameter(torch.empty(rank, output_dim)) # LoRA weight B
```

```
# Initialization of LoRA weights
```

```
nn.init.kaiming_uniform_(W_A, a=math.sqrt(5))
nn.init.zeros_(W_B)
```

```
def regular_forward_matmul(x, W):
    h = x @ W
    return h
```

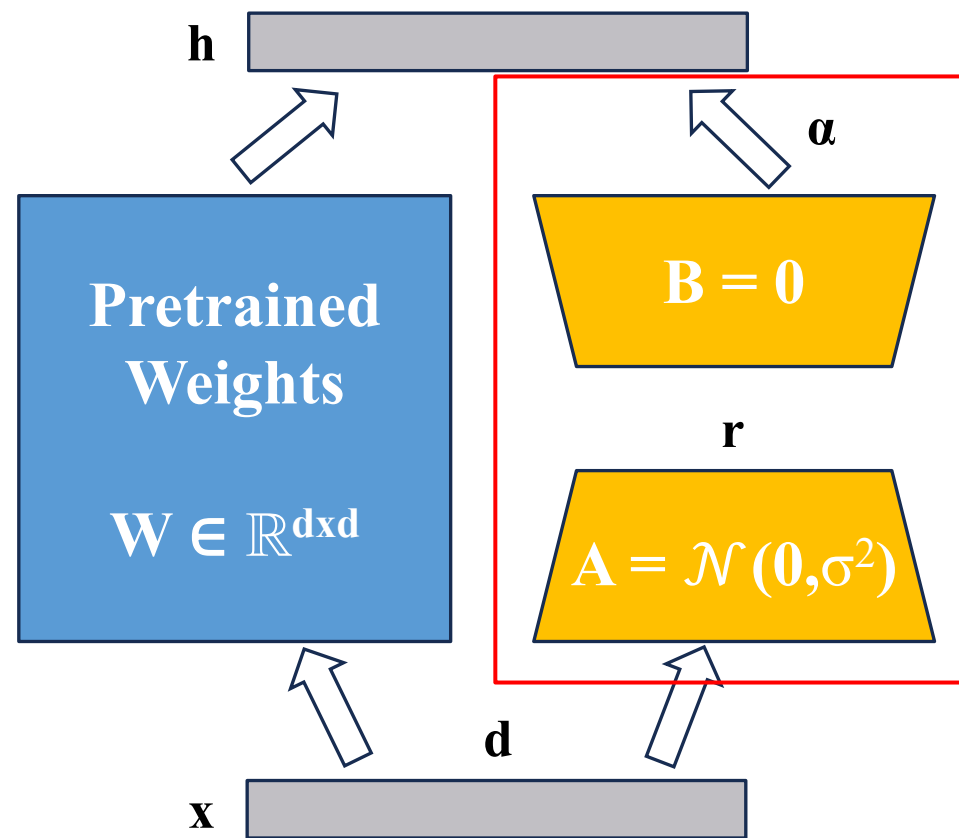
```
def lora_forward_matmul(x, W, W_A, W_B):
    h = x @ W # regular matrix multiplication
    h += x @ (W_A @ W_B) * alpha # use scaled LoRA weights
    return h
```





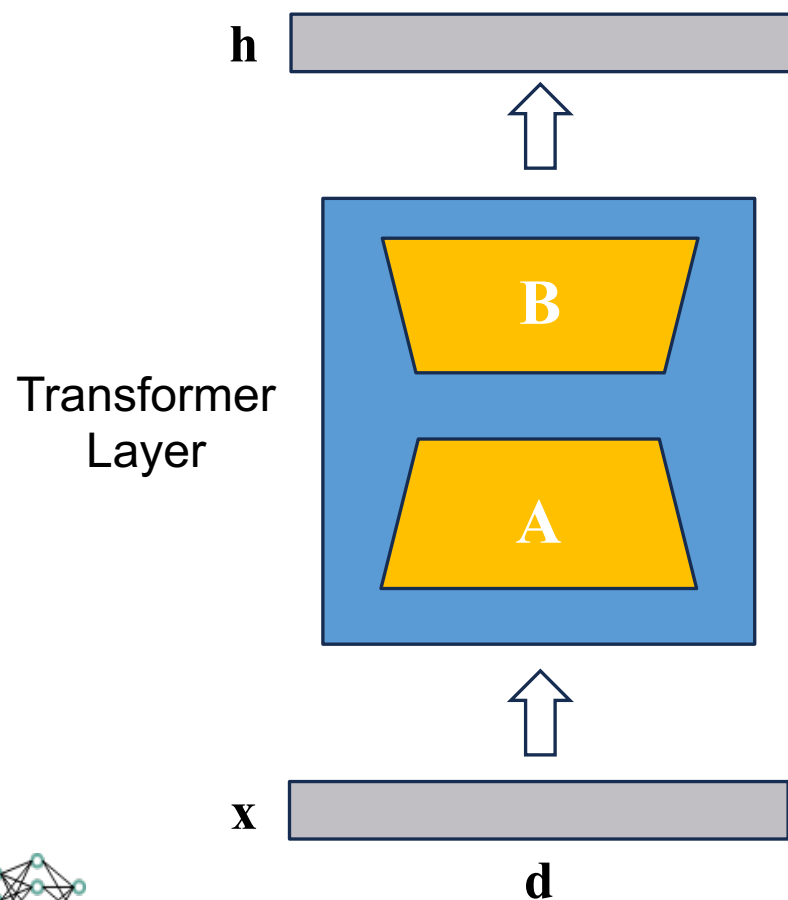
# [注意事項] LoRA 的問題

- $r$  的數值大小需要手動調整
  - $r$  越小，訓練參數量越少
- 相較於原本沒有 LoRA 的模型，  
LoRA 其實會讓 inference 速度變慢



# 可不可以不要外加？(Adapter 作法)

**Adapter:** Parameter-Efficient Transfer Learning for NLP. ICML 2019



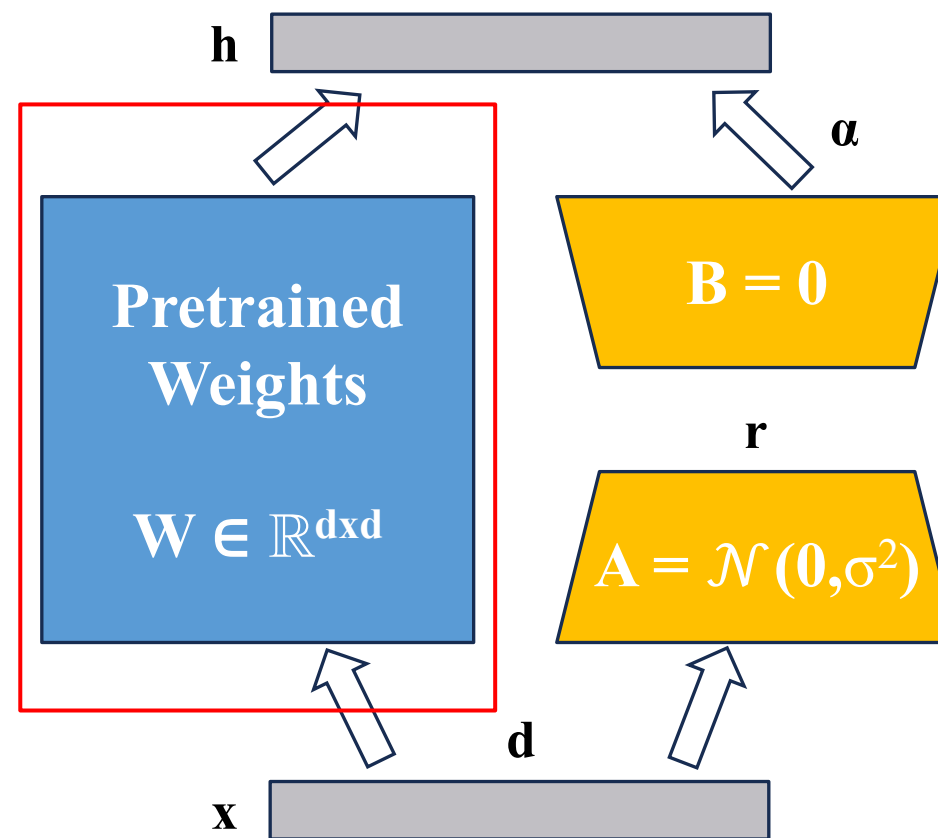
優點：模型設計更靈活

缺點：

- 直接變更了模型架構 (串聯)
  - -> 計算時間比 LoRA 長

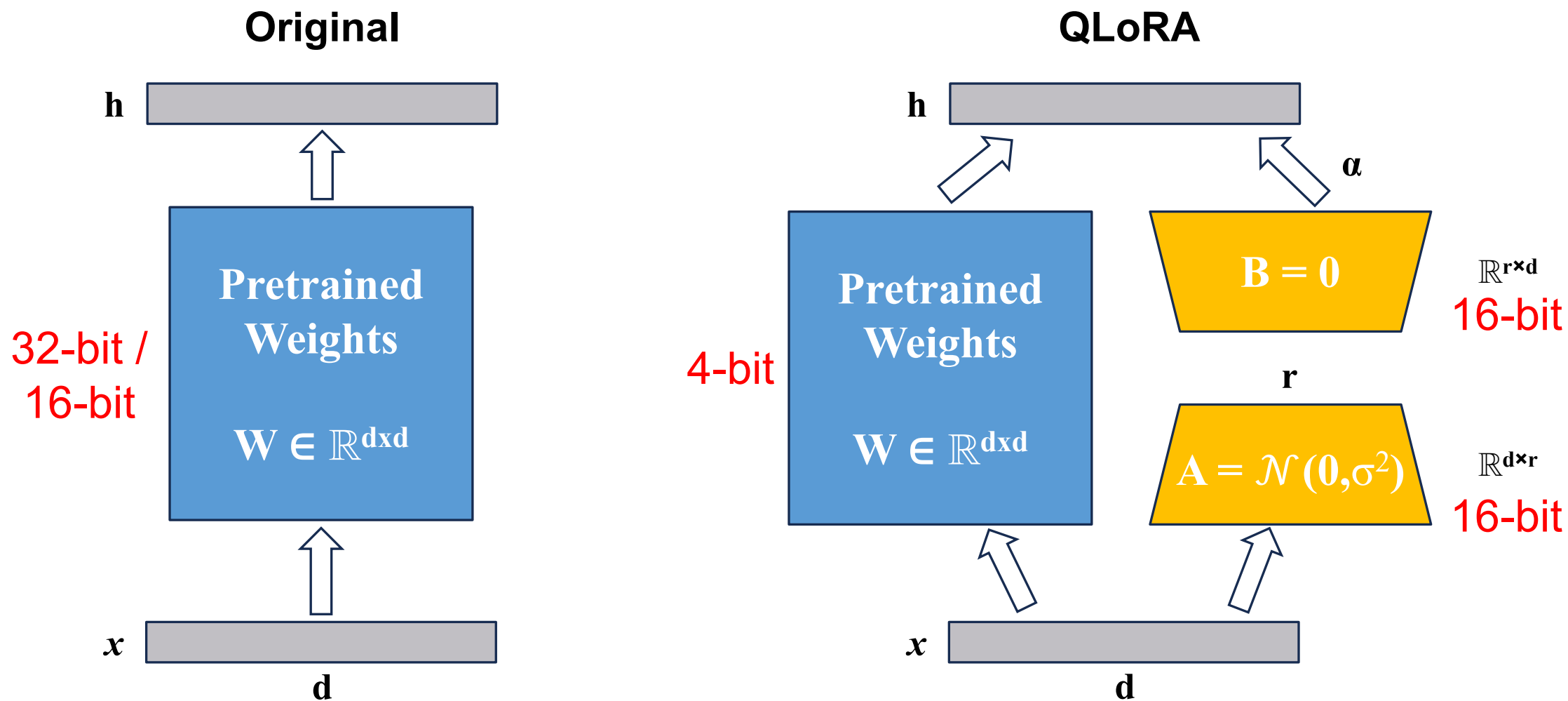
# [注意事項] LoRA 的問題

- $r$  的數值大小需要手動調整
  - $r$  越小，訓練參數量越少
- 相較於原本沒有 LoRA 的模型，LoRA 其實會讓 inference 速度變慢
- 訓練和 inference 時，大模型的部分其實還是被載入記憶體...



# QLoRA: **Quantized** LoRA

只有黃色部分會被訓練  
 $\alpha$  是超參數



# [注意事項] QLoRA

---

- 記憶體使用量： $\text{QLoRA} < \text{LoRA}$
- 模型效能： $\text{QLoRA} \leq \text{LoRA}$



# Quantization 方式

---

 Post-training Quantization  
(PTQ)

在模型訓練好之後，直接將權重轉成低精度，  
不重新訓練

 Quantization-aware Training  
(QAT)

在訓練過程中就模擬量化的影響，讓模型學會適應低精度



# PTQ 與 QAT 比較

|         |  PTQ |  QAT |
|---------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 何時做量化？  | 模型訓練完之後                                                                               | 訓練過程中就模擬量化                                                                              |
| 需不需再訓練？ | ❌ 有 pre-trained model 就可                                                              | ✅                                                                                       |
| 最後模型表現  | 較差 (轉換後精度越低，表現越差)                                                                     | 較好                                                                                      |
| 實作難易度   | 較簡單                                                                                   | 較難                                                                                      |



# Gemma 3 QAT

GEMMA / AI EDGE

## Gemma 3 QAT Models: Bringing state-of-the-Art AI to consumer GPUs

APRIL 18, 2025

Edouard YVINEC  
Research Scientist

Phil Culliton  
ML Engineer

Share

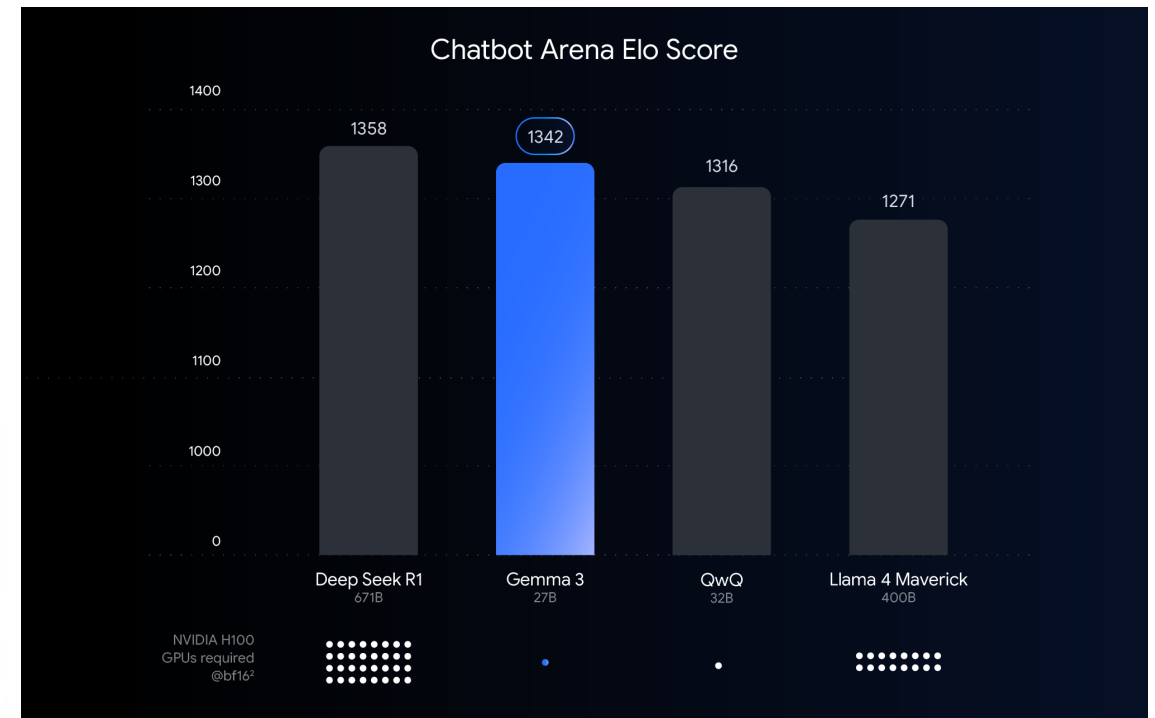


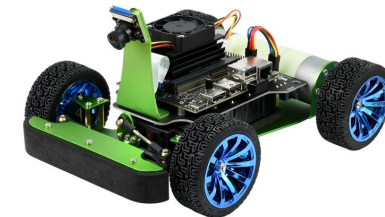
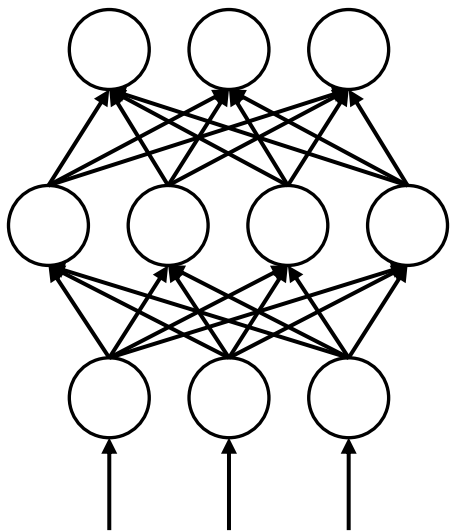
Figure source:  
<https://developers.googleblog.com/en/gemma-3-quantized-aware-trained-state-of-the-art-ai-to-consumer-gpus/>





# How about model compression?

---



# Comparison: What is Model Compression?

---



**Original Model**



**Quantized Model**



**Pruned Model**



**Distilled Model**



# PEFT vs. Model compression

|                  | PEFT                  | Model Compression    |
|------------------|-----------------------|----------------------|
| 目標               | 讓模型適應新的任務，<br>但模型大小不變 | 加速模型運算或模型儲存空間        |
| 相較於原始模型的<br>改變內容 | 插入少量可訓練參數             | 減少整體模型結構或權重          |
| 參數更新             | 只更新少量新參數              | 先減少整體模型結構後針對新的模型進行訓練 |
| 使用情境             | 需要模型學會新的任務時           | 手機、邊緣裝置              |



# Previous Videos of Model Compression

---

- [https://youtu.be/Qqyo9U2JRbQ?si=V9F\\_nNm44dijmYKc](https://youtu.be/Qqyo9U2JRbQ?si=V9F_nNm44dijmYKc)
- <https://youtu.be/luBa01XYoBc?si=8Hj8fUUI6RwPPI1j>
- <https://youtu.be/dpE6LyIGU9w?si=5XBdr-JDJaHojoJD>



# Announcements

---

# Project checkpoints

---

- ~~Week 9: 確定各組的題目~~
- ~~Week 12: 進度報告 PPT (5 pages)~~
- Week 14: 進度報告 PPT (5+5 pages), Presentations (selected teams)
- Week 16: Final presentations for all teams
- Week 16-17: 繳交書面報告以及程式碼



# Week 14 之前要繳交什麼？

---

- 前5頁：Checkpoint1 原始簡報內容 (如有需要，可修改，不需要與Checkpoint1完全一樣)
- 後5頁 (或更多)：新進度補充
  1. 實作的方法介紹 (代表各組需完成初步實作)，可以包含：
    - 資料前處理、模型介紹、訓練策略 (如 loss function、optimizer、scheduler 等) 等...
  2. 實驗結果比較 (含實驗設定說明)
  3. Kaggle Leaderboard 名次 (請截圖貼到pptx中)
  4. 時程規劃 (再來還要測試什麼？用表格列出未來 1-2 週內的預定測試與安排)
  5. 針對 Checkpoint2 之前的小組分工細節



# Checkpoint 2 繳交注意事項

---

- 一組繳交一份，請上傳至 **E-Learning**
- 檔名：DL\_teamN\_checkpoint2.pdf 或 DL\_teamN\_checkpoint2.pptx
- Deadline:
  - **2025/12/02 星期二 23:59 前**
  - (如果你想要 Week 14 進行口頭報告) -> **2025/12/01 星期一 18:00 前**
    - 將於 2025/12/02 星期二中午 **12:00** 前公布 Week 14 進行報告的組別
    - 報告內容同前一頁的項目規範，可以自行精簡





# Project 各個階段分數佔比

Final Project 佔學期總成績 40%

| 查核點 (週次)                 | 對象: 繳交內容                                                                           | 分數佔比                                      |
|--------------------------|------------------------------------------------------------------------------------|-------------------------------------------|
| Checkpoint1 (Week 12)    | All teams: 進度報告 PPT (5 pages)檔案                                                    | 5%                                        |
| Checkpoint2 (Week 14)    | All teams: 進度報告 PPT (5+5 pages*)檔案<br>Selected teams: 取6組 (1題目2組) 於課堂中報告, 1組 10min | 5% W14 報告組別可得 Checkpoint1 和 Checkpoint2滿分 |
| Checkpoint3 (Week 16)    | All teams: 最終口頭報告                                                                  | 15%                                       |
| Checkpoint4 (Week 16-17) | All teams: 書面報告檔案 (含程式碼)、小平台 (optional)                                            | 15%                                       |

\*繼承Checkpoint1內容+實作



# Thank you!

Instructor: 林英嘉

 yjlin@cgu.edu.tw

TA: 劉美辰

 m1461014@cgu.edu.tw